

Die Verarbeitung von Ereignissen und das Erkennen von Ereignismustern bilden den Kern des Complex Event Processing. Dieses Kapitel führt die wesentlichen Sprachkonzepte der Ereignisverarbeitung anhand einer vereinfachten Beispielsprache zur Formulierung von Ereignisregeln ein – differenziert nach dem *Bedingungs-
teil* mit der Beschreibung von Ereignismustern (Abschn. 3.1) und dem *Aktionsteil* (Abschn. 3.2). Der anschließende Abschn. 3.3 stellt mit Esper eine konkrete EPL vor.

3.1 Konzepte zur Beschreibung von Ereignismustern

Genauso wie sich in der Objektorientierung die Konzepte *Klasse* und *Objekt* unterscheiden lassen, kann in der Ereignisverarbeitung zwischen *Ereignistypen* und *Ereignisinstanzen* differenziert werden.¹

Ereignistyp: Ein *Ereignistyp* (event type, event class) spezifiziert die grundlegenden Eigenschaften einer ganzen *Klasse* von *gleichartigen* Ereignissen [18]. Ein Ereignistyp definiert insbesondere die *Attribute* (event properties) einer Klasse von Ereignissen, so dass alle Ereignisse eines bestimmten Typs dieselbe Struktur der in ihnen enthaltenen Daten besitzen.

¹ Häufig wird nicht präzise zwischen Ereignistypen und Ereignisinstanzen unterschieden, sondern einfach der Begriff *Ereignis* verwendet. Auch in diesem Buch wird der Einfachheit halber von Ereignissen gesprochen, wenn der Bezug klar ist.

Beispielsweise enthält ein Ereignis vom Typ ‚Kursänderung eines Wertpapiers‘ eine eindeutige Identifikationsnummer (ID), einen Zeitstempel, das Wertpapier-symbol sowie den Wert des aktuellen Kurses: Ereignistyp `StockEvent` mit den Attributen `ID`, `timestamp`, `symbol` und `price`.

Ereignisinstanz bzw. Ereignisobjekt: Eine *Ereignisinstanz* (event instance) bzw. ein *Ereignisobjekt* (event object) beschreibt ein *konkretes* Vorkommnis mit seinen kennzeichnenden Daten, d. h. es repräsentiert eine Ausprägung eines Ereignistyps.

Bezogen auf das obige Beispiel besitzt eine Ereignisinstanz die Daten für eine konkrete Kursänderung eines Wertpapiers, beispielsweise für eine IBM-Aktie:

```
1 StockEvent(ID=4711, timestamp=1264123099, symbol=ibm, price=333)
```

Die bislang aufgetretenen Ereignisinstanzen bilden einen kontinuierlichen Ereignisstrom. Ein definierter Ausschnitt aus dem potenziell unendlichen Ereignisstrom wird als *Ereignisfolge* bezeichnet.

Ereignisfolge: Eine *Ereignisfolge* ist eine *Sequenz von Ereignisinstanzen* (event instance sequence) und kann folgendermaßen notiert werden: $a_1b_1b_2c_1a_2b_3$. Dabei bezeichnet a_i eine Ereignisinstanz vom Typ A und der Index i steht für das i -te Auftreten eines Ereignisses dieses Typs.

Die Ereignisinstanz a_2 ist also das zweite Ereignis vom Typ A in einer Ereignisfolge. Im Folgenden werden Ereignisinstanzen durch Kleinbuchstaben und Ereignistypen durch Großbuchstaben bezeichnet.

Der *Bedingungsteil* einer Ereignisregel setzt sich aus einem oder mehreren verknüpften *Ereignismustern* zusammen. Ein Ereignismuster beschreibt eine bestimmte Situation in einer Ereignisfolge, die von besonderem Interesse ist und die dementsprechend erkannt werden soll.

Ereignismuster: Ein *Ereignismuster* (event pattern) legt *Bedingungen* an eine Ereignisfolge fest. Die Spezifikation der Bedingungen erfolgt mithilfe der Operatoren einer *Ereignisalgebra*. Eine Folge von Ereignisinstanzen *erfüllt* ein Muster, wenn sie die im Muster festgelegten Bedingungen erfüllt [18].

Beispielsweise erfüllt die Ereignisfolge b_2c_1 das Muster „auf eine Ereignisinstanz vom Typ B folgt eine Instanz vom Typ C “.

Spezifikation von Ereignismustern

Eine *Ereignisalgebra* definiert Bedingungen, d. h. Beziehungen und Einschränkungen, für eine Menge von Ereignissen. Dabei sollen fachliche, zeitliche und kausale Zusammenhänge zwischen Ereignissen spezifiziert werden können. Als Grundlage eines Ereignismusters gilt es zunächst, die Sequenz der erwarteten Ereignistypen festzulegen. Eine solche Sequenz lässt sich mithilfe spezieller Operatoren der Ereignisalgebra beschreiben.

Ereignisalgebra (I): Ein **ereignistypbasiertes Muster** (event type pattern) definiert die am Muster beteiligten Ereignistypen. Folgende Operatoren definieren Abhängigkeiten zwischen Ereignistypen:

- Der **Sequenzoperator** $A \rightarrow B$ legt die zeitliche Reihenfolge fest, in der Ereignisinstanzen der definierten Typen auftreten müssen. Im Beispiel muss eine Ereignisinstanz vom Typ B einer Instanz vom Typ A folgen. Dazwischen können durchaus Ereignisse anderer Typen auftreten. Zum Beispiel erfüllt die Ereignisfolge $a_1c_1d_1c_2b_1$ das Muster $A \rightarrow B$.
- Die **booleschen Operatoren** $\wedge$ und $\vee$ definieren unabhängig von der Reihenfolge, welche Typen von zwei Ereignisinstanzen auftreten müssen. $A \wedge B$ bedeutet, dass ein Ereignis vom Typ A *und* eins vom Typ B vorliegen müssen. $A \vee B$ erfordert entweder ein Ereignis vom Typ A *oder* eins vom Typ B .
- Der **Negationsoperator** $\neg A$ beschreibt, dass *keine* Instanz von einem bestimmten Ereignistyp (hier A) auftreten darf. Dieser Operator ist nur sinnvoll, wenn er sich auf ein bestimmtes Betrachtungsintervall bezieht (siehe nachfolgend den Abschnitt über *Sliding Windows*).

Die folgenden Beispiele verdeutlichen, wie die verschiedenen Operatoren kombiniert werden können, um komplexe Muster zwischen Ereignistypen auszudrücken. Das Muster

$$1 \quad A \rightarrow (B \vee C)$$

sucht nach einer Ereignisfolge, in der nach einem Ereignis vom Typ A ein Ereignis vom Typ B oder vom Typ C auftritt. Beispielsweise erfüllen die Ereignisfolgen $a_1d_1e_1b_1$ oder $a_1d_1c_1$ dieses Muster. Das Ereignismuster

$$1 \quad (A \rightarrow \neg B \rightarrow C)$$

erfordert eine Ereignisfolge, in der ein Ereignis vom Typ C einem Ereignis vom Typ A folgt, ohne dass ein Ereignis vom Typ B dazwischen liegt. Somit erfüllt die Folge $a_1d_1c_2$ das Muster, aber $a_1b_1d_1c_2$ hingegen nicht.

In vielen Fällen reicht es nicht aus, in einem Muster nur die Ereignistypen mit den entsprechenden Reihenfolgebedingungen festzulegen. Vielmehr ist es erforderlich, auch den *Kontext* der Ereignisse genauer zu beschreiben.

Ereignisalgebra (II): Kontextbedingungen (context conditions) definieren Anforderungen an die *Attributwerte* einer Ereignisinstanz. Um Ereignisinstanzen in Mustern verwenden zu können, müssen folgende Sprachkonstrukte eingeführt werden:

- **Aliasnamen** für Ereignisinstanzen ermöglichen es, in Bedingungen verschiedene *Instanzen* zu verwenden. Aliasnamen werden mit dem Schlüsselwort **AS** definiert.
- Der „**-Operator**“ beschreibt den Zugriff auf ein Ereignisattribut.
- Mithilfe von *Daten aus der Anwendungsdomäne* lassen sich weitere Kontextbedingungen formulieren. Der Zugriff auf Anwendungswissen ist z. B. durch Methodenaufrufe in Regeln möglich.

Für die Beschreibung von Bedingungen können die üblichen, für die jeweiligen Datentypen definierten Operationen und Vergleichsoperatoren verwendet werden. Für numerische Datentypen sind dies beispielsweise $+$, $-$, $/$, $*$, $<$, $>$, $\leq$, $\geq$, $=$, $\neq$ usw.

Die folgenden Beispiele zeigen, wie diese Sprachkonstrukte eingesetzt werden können. Beispielsweise sucht das Ereignismuster

```
1  ((A AS a) → (B AS b)) ∧ (a.attribute1 = b.attribute2)
```

eine Ereignisinstanz vom Typ *A*, gefolgt von einer Instanz vom Typ *B*, deren Attribute *attribute1* und *attribute2* mit identischen Werten belegt sind.

Ein Muster, das Daten aus der Anwendungsdomäne verwendet, ist:

```
1  SensorEvent AS s
2      ∧ s.temp < (0.5 * Sensorprop.getMaxTemp(s.type))
```

Gesucht werden Ereignisse vom Typ *SensorEvent* mit einem Temperaturwert, der weniger als 50 % der maximalen Temperatur beträgt, die für diese Art von Sensor zugelassen ist. Im Beispiel stellt die Anwendungsklasse *Sensorprop* die Methode *getMaxTemp()* zur Abfrage der maximal erlaubten Temperatur bereit.

Sliding Windows

Theoretisch können Ereignismuster sämtliche zuvor aufgetretenen Ereignisse berücksichtigen. Da jedoch kontinuierlich neue Ereignisse entstehen, liegt eine ständig wachsende und damit quasi unbegrenzt große Menge von Ereignissen vor, zum Beispiel alle Aktienkurse der zurückliegenden Jahre.

Um den Abgleich der Ereignisse mit einem Muster effizient – und möglichst nahezu in Echtzeit – zu erreichen, wird deshalb die Anzahl der betrachteten Ereignisse auf Ausschnitte aus der Ereignismenge beschränkt. Diese Ausschnitte werden durch sogenannte *Sliding Windows* beschrieben.

Ereignisalgebra (III): Sliding Windows beschränken die Anzahl der betrachteten Ereignisse in einem Ereignisstrom. Dabei lassen sich Längen- und Zeitfenster unterscheiden.

1. Ein *Längenfenster* (*length window*) der Länge *n* berücksichtigt nur die letzten *n* Ereignisse des Ereignisstroms.
 2. Ein *Zeitfenster* (*time window*) betrachtet nur diejenigen Ereignisse, die im definierten Zeitraum, z. B. in den letzten 5 Minuten, aufgetreten sind.
- Sliding Windows werden durch entsprechende Schlüsselwörter **win:length:x** bzw. **win:time:y** in eckigen Klammern spezifiziert.

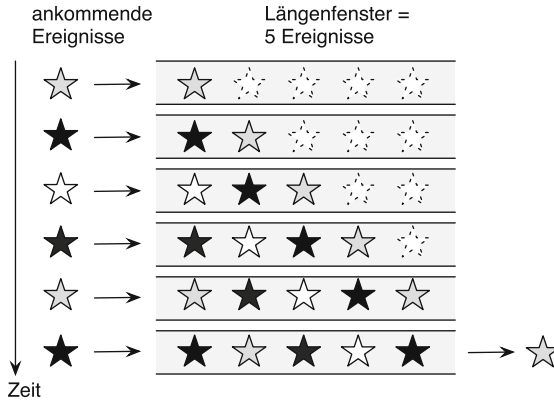


Abb. 3.1 Sliding Window – Längenfenster mit einem Puffer der Länge 5

Die Abb. 3.1 zeigt ein Längenfenster mit einem Ereignispuffer der Länge 5. Bei Ankunft eines neuen Ereignisses wird das älteste Ereignis gemäß der FIFO-Strategie (*First-In-First-Out*) aus dem Puffer geschoben.

Als Beispiel für ein Zeitfenster beschreibt das Muster $A[\text{win:time:5min}]$ die Menge der Ereignisinstanzen vom Typ A in einem Zeitintervall der letzten 5 Minuten. Im Beispielmuster

1 $(A \rightarrow B)[\text{win:time:5min}] \rightarrow C$

muss innerhalb eines Zeitintervalls von 5 Minuten ein Ereignis vom Typ B auf ein Ereignis vom Typ A folgen; anschließend wird zu einem beliebigen Zeitpunkt noch ein drittes Ereignis vom Typ C erwartet.

Die Abb. 3.2 stellt die Ereignisfolge bei einem Zeitfenster mit der Intervalllänge von 3 Zeiteinheiten dar. Jedes Ereignis besitzt eine bestimmte *Gültigkeitsdauer*, nach deren Ablauf es aus dem Fenster der betrachteten Ereignisse entfernt wird, im Beispiel der Abbildung geschieht dies nach 3 Zeiteinheiten. Die Abbildung zeigt, dass das Zeitfenster unterschiedlich viele Ereignisinstanzen enthalten kann.

Aggregationen

Das Konzept der Sliding Windows dient nicht nur dazu, die Menge der Ereignisse einzuschränken, sondern ermöglicht es auch, Ereignisse zeitlich zusammenzufassen. Dazu stehen entsprechende *Aggregationsfunktionen* zur Verfügung.

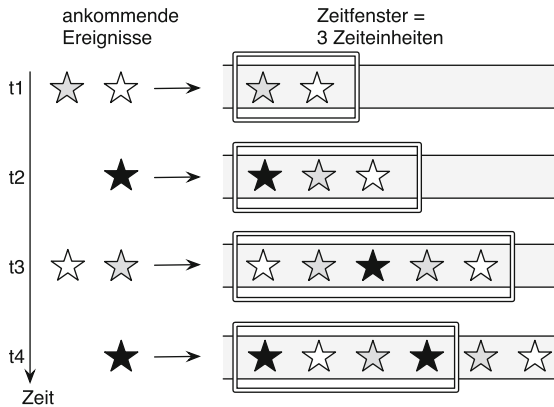


Abb. 3.2 Sliding Window – Zeitfenster mit einer Länge von 3 Zeiteinheiten

Ereignisalgebra (IV): Aggregationsfunktionen fassen die Attributwerte einer definierten Menge von Ereignissen zusammen. Typische Aggregationsfunktionen sind:

- **sum()**: bildet die Summe eines Attributwertes über alle Ereignisse
- **avg()**: berechnet den Durchschnittswert eines Attributwertes
- **min()** und **max()**: liefern das Minimum bzw. Maximum des Wertes

Durch die Verwendung von Aliasnamen lassen sich die Resultate von Aggregationsfunktionen innerhalb einer Regel verwenden. Das folgende Ereignismuster berechnet den Durchschnitt des Attributs `price` für alle `StockEvent`-Ereignisse der letzten 5 Minuten.

```
1 StockEvent.avg(price)[win:time:5min] AS averagePrice
```

Bei der bisherigen Betrachtung bleibt allerdings die Frage offen, *wie oft* die Durchschnittsberechnung im obigen Beispiel durchgeführt wird. Dabei gibt es zwei verschiedene Möglichkeiten:

1. Jedes Eintreffen eines `StockEvent`-Ereignisses löst eine erneute Berechnung aus.
2. Die Berechnung erfolgt erst nach Ablauf des Sliding Windows, d. h., für ein Zeitfenster von 5 Minuten erfolgt die Berechnung alle 5 Minuten, für ein Längenfenster der Größe 10 nach Eintreffen von 10 Ereignissen. In diesem Fall kann das zugehörige Sliding Window als *Batch-Window* (Schlüsselwort **batch**) spezifiziert werden. Das obige Beispielmuster lautet dann für ein Batch-Window:

```
1 StockEvent.avg(price)[win:time:batch:5min] AS averagePrice
```

3.2 Spezifikation von Aktionen in Ereignisregeln

Enthält der einfließende Ereignisstrom eine Ereigniskonstellation, die das im Bedingungsteil einer Ereignisregel definierte Muster erfüllt, so wird der *Aktionsteil* der erfüllten Regel angestoßen.

Ereignisregel: Eine *Ereignisregel* setzt ein *Muster* $P()$ und eine *Aktion* $A()$ zueinander in Beziehung und lässt sich in allgemeiner Form durch

CONDITION $P(e_1, e_2, \dots, e_n)$ **ACTION** $A(e_1, e_2, \dots, e_n)$

beschreiben. Dabei ist $e_1, e_2, \dots, e_n$ eine zeitlich geordnete Ereignisfolge mit beliebigen Ereignistypen, die das Muster P erfüllt.

Die *Mustererkennung* löst eine *Aktion* $A(e_1, e_2, \dots, e_n)$ mit dem Auftreten des letzten Ereignisses e_n aus, das für ein Muster benötigt wird.

Aktion: Eine *Aktion* $A(e_1, e_2, \dots, e_n)$ realisiert die *Ereignisbehandlung* und ist für eine angemessene Reaktion auf die eingetretene Ereignissituation verantwortlich. Dazu benötigt sie den *Aktionskontext*, weshalb ihr die auslösende Ereignisfolge $(e_1, e_2, \dots, e_n)$ als Parameter übergeben wird.

Der Aktionskontext beschreibt zeitliche, örtliche und weitere semantische Eigenschaften der für das Auslösen der Aktion verantwortlichen Ereignisse. Im Wesentlichen lassen sich zwei *Arten von Aktionen* unterscheiden:

1. *Erzeugen neuer Ereignisse*

Die Aktion erzeugt ein *neues Ereignis*, das entweder nur die Daten der auslösenden Ereignisse aufbereitet oder die durch das Eintreten des Musters gewonnenen Erkenntnisse beschreibt. Das neu erzeugte Ereignis fließt in Form einer Rückkopplung an die CEP-Komponente zurück und wird dort weiterverarbeitet.

2. *Anstoßen von Diensten*

Die Aktion *stößt einen Dienst* in einem nachgelagerten Anwendungssystem an. Dort führen die operativen Systeme eine entsprechende Geschäftsaktion aus.

1. Erzeugen neuer Ereignisse

Das Erzeugen neuer Ereignisinstanzen innerhalb einer Regel kann als *Ereignistransformation* verstanden werden. Die Ereignisfolge $e_1, e_2, \dots, e_n$, die das Muster erfüllt hat, wird in eine oder mehrere neue Ereignisinstanzen transformiert. Entsprechend muss es im Aktionsteil einer Ereignisregel ein Sprachkonstrukt `create` zum Erzeugen einer neuen Ereignisinstanz geben. So wird beispielsweise durch

```
1 create BuyEvent (share=ibm, limit=5000, currency=Euro)
```

eine neue Ereignisinstanz vom Typ `BuyEvent` erzeugt, die den möglichen Kauf einer IBM-Aktie für 5000 Euro anzeigt.

Verschiedene grundlegende Transformationsschritte lassen sich unterscheiden:

1. Transformationen, die die Ereignisfolge zwar umwandeln, aber keine neuen Informationen hinzufügen:
 - (a) *Translation*, (b) *Filterung* und (c) *Aggregation*
2. Transformationen, bei denen die neu erzeugten Ereignisse mehr Informationen mit sich führen als die ursprüngliche Ereignisfolge:
 - (d) *Anreicherung mit Kontextwissen* und (e) *Synthese komplexer Ereignisse*

(a) Translation: Ein *Übersetzungsschritt* überführt die Daten einer Ereignisinstanz in eine Instanz eines anderen Ereignistypen.

Ein Standardschritt in der Ereignisverarbeitung ist die Abbildung von Ereignisinstanzen eines Typs *A* auf einen anderen Typ *B*. Beispielsweise um Ereignisse aus verschiedenen heterogenen Quellen, wie z. B. Sensoren oder Softwaresystemen, in ein einheitliches Format zu überführen.

(b) Filterung (filtering): Um mit der riesigen Menge der erzeugten Ereignisse umgehen zu können, werden die für eine fachliche Fragestellung relevanten Ereignisse aus einer Ereignisfolge *herausgefiltert*.

Das Filtern reduziert die Anzahl der betrachteten Ereignisse und ist ein wesentliches Konzept für die Effizienzsteigerung der Ereignisverarbeitung.

Beispielsweise sollen in einem Wertpapierhandelssystem nur die Ereignisse, die Kurswerte mit einem Wert größer als 500 Euro repräsentieren, betrachtet werden. Entsprechend filtert die folgende Ereignisregel alle Ereignisse mit einem größeren Kurswert aus dem Ereignisstrom heraus:

```

1 CONDITION ( StockEvent AS s )
2           ^ s.price > 500
3 ACTION create RelevantStockEvent ( symbol=s.symbol , price=s.price )

```

Den *CONDITION*-Teil der Regel erfüllen nur *StockEvent*-Ereignisse mit einem Betrag größer als 500 – dieses Ereignismuster implementiert das Filtern der Ereignisse. Der *ACTION*-Teil erzeugt dann eine neue Instanz des Ereignistyps *RelevantStockEvent* und initialisiert diese Instanz mit den Attributwerten des *StockEvent*-Ereignisses.²

Eine weitere Möglichkeit zur Reduktion der Ereignismenge bietet die Aggregation von Ereignissen.

² Genau genommen filtert die obige Regel keine Ereignisse aus dem Ereignisstrom heraus, sondern selektiert die sinnvollen Ereignisse und kopiert sie in einen neuen Ereignistyp. Einige EPLs bieten überhaupt keinen Befehl zum *Löschen* von Ereignisinstanzen. Das entspricht der Sichtweise, dass Ereignisse nicht ungeschehen gemacht werden können. In diesem Fall kann, wie im obigen Beispiel gezeigt, ein eigener Ereignistyp für die gefilterten Ereignisse eingeführt werden.

(c) **Aggregation:** Eine *Aggregation* fasst mehrere einfache Ereignisse zusammen, um aussagekräftige Daten zu erhalten. Hierbei kommen u.a. die Aggregationsfunktionen aus Abschn. 3.1 zur Anwendung. Insbesondere betrifft die Zusammenfassung oft zeitlich oder räumlich korrelierte Ereignisse.

Die folgende Regel erzeugt Ereignisse, die den Durchschnittswert einer Aktie in den letzten 200 Tagen berechnen.

```

1 CONDITION ( StockEvent AS s ) [ win : time : 200 days ]
2             ^ s . symbol = ibm
3             ^ s . avg ( price ) AS average
4 ACTION create 200DayAvgEvent ( symbol = s . symbol , value = average )

```

Die Regel betrachtet in einem Zeitfenster von 200 Tagen jedes ankommende `StockEvent` mit dem angegebenen Aktiennamen³ und berechnet den Durchschnittswert des Attributs `price`. Schließlich erzeugt die Regel im *ACTION*-Teil eine Ereignisinstanz vom Typ `200DayAvgEvent` mit dem Aktiennamen des Ausgangsereignisses und dem kalkulierten Durchschnittswert.

Für die Spezifikation von Ereignisregeln werden häufig Informationen benötigt, die nicht unmittelbar in den Ereignissen enthalten sind, sondern in den angebundenen Anwendungssystemen zur Verfügung stehen. Durch die Anreicherung der Ereignisinstanzen mit Kontextwissen werden die Instanzen selbst mit den für ihre Weiterverarbeitung erforderlichen Informationen versorgt.

(d) **Anreicherung mit Kontextwissen** (content enrichment): Im *Content-Enrichment* werden Ereignisinstanzen mit *zusätzlichen Daten* aus den Anwendungssystemen angereichert.

Es existieren viele Beispielfälle, in denen die Ergänzung von Ereignisdaten durch Anwendungsdaten notwendig ist. Beispielsweise werden häufig eindeutige, aber

³ Aus Gründen der Vereinfachung ist in der Regel der Aktienname als fester Wert angegeben. Natürlich sollte in einer realen Anwendung der Name der Aktie als Parameter übergeben werden.

fachlich wenig aussagekräftige Identifikationsnummern (IDs) – wie Personal- oder Artikelnummern – durch detailliertere Personal- oder Produktdaten ergänzt. Anschließend können Ereignismuster auch auf diese Attribute unmittelbar zugreifen.

Die folgende Regel zeigt beispielhaft, wie ein technisches Messereignis eines Sensors vom Typ `MeasuringEvent` um Anwendungswissen ergänzt werden kann.

```

1 CONDITION ( MeasuringEvent AS m)
2 ACTION create AirPollutionEvent( senorID=m.sensorID , value=m.value ,
3                               geopos=GeoService.getPos(m.sensorID ))
```

Für Messereignisse von Sensoren ist es in der Regel wichtig festzustellen, an welchem geographischen Standort der Messwert ermittelt wurde. Die von der Anwendungsklasse `GeoService` bereitgestellte Methode `getPos()` liefert zu einer Sensor-ID dessen Standortkoordinaten zurück.

In den obigen Verarbeitungsschritten werden die Ereignisse nur sehr moderat geändert. Eine Schlüsseleigenschaft für die Mächtigkeit von Complex Event Processing liegt aber darin, aus der Masse der aufgetretenen Ereignisse gänzlich neue Rückschlüsse und Erkenntnisse zu folgern, und zwar durch die Ableitung von *komplexen* Ereignissen [12].

(e) Synthese komplexer Ereignisse (synthesis of complex events): Neue komplexe Ereignisse entstehen als *Abstraktion* der in einem Ereignismuster in Beziehung gesetzten einfachen Ereignisse. Muster zur Erzeugung komplexer Ereignisse stellen kausale, temporale, räumliche und sonstige semantische Beziehungen zwischen Ereignissen her, aus denen sich eine fachliche Bedeutung ableiten lässt.

Beispielsweise wird zur Erkennung eines Phishing-Versuchs⁴ beim Online-Banking nach auffälligen Verhaltensmustern gesucht. Ein solches ungewöhnliches Verhaltensmuster liegt zum Beispiel vor, wenn unmittelbar nach der Passwortänderung eines Online-Bankkontos ein hoher Geldbetrag von genau diesem Konto abgehoben wird. Die folgende Regel beschreibt ein solches Verhaltensmuster.

⁴ Phishing (= password fishing) bezeichnet das unrechtmäßige Ausspähen von Zugangsdaten für Internetanwendungen.

```

1 CONDITION ( PasswordChangeEvent AS p → WithdrawEvent AS w)
2                                     [ win : time : 15 min ]
3         ∧ p.account=w.account
4         ∧ w.amount>10.000 Euro
5 ACTION   create PhishingEvent(account=p.account)

```

Das Ereignismuster sucht Situationen innerhalb des Ereignisstroms, in denen für dasselbe Konto ein `WithdrawEvent` auf ein `PasswordChangeEvent` innerhalb eines Zeitfensters von 15 Minuten folgt.

An diesem Beispiel lässt sich sehr deutlich der *Erkenntnisgewinn* durch komplexe Ereignisse erkennen: Es werden nicht nur arithmetische Berechnungen durchgeführt, sondern eine auf Erfahrungen basierende *Schlussfolgerung* abgeleitet. Wenn nach einer erfolgreichen (aber unrechtmäßigen) Änderung des Passworts anschließend sofort ein großer Geldbetrag abgehoben wird, leitet die Regel die Schlussfolgerung ab, dass es sich bei diesem Vorgang um einen Betrugsfall handeln könnte. Das neue komplexe Ereignis vom Typ `PhishingEvent` beschreibt die Situation auf einer höheren Abstraktionsebene als die beiden einfachen Ereignisse.

2. Anstoßen von Diensten

Für die eigentliche Ereignisbehandlung sind in der Regel die nachgelagerten Unternehmensanwendungen verantwortlich, in denen die operativen Geschäftsprozesse realisiert sind (siehe Abb. 2.2). Neben der Erzeugung neuer Ereignisse kann deshalb im Aktionsteil einer Regel auch ein *Dienst* eines Anwendungssystems *ausgelöst* werden.⁵ Insbesondere sollen am Ende der Ereignisverarbeitungskette die mittels CEP gewonnenen Erkenntnisse unmittelbar in die operativen Unternehmensanwendungen einfließen.

Technisch gibt es unterschiedliche Möglichkeiten für das Anstoßen eines Dienstes aus einer Ereignisregel heraus:

- Durch das Senden einer *Nachricht* (message) über eine nachrichtenorientierte Middleware (MOM). Nachgelagerte Anwendungssysteme können dann auf

⁵ Der Begriff *Dienst* ist in diesem Kontext als Oberbegriff für jedwede Art von Geschäftsabläufen bzw. -funktionalität zu verstehen, also Methoden, Funktionen, Abläufe in Workflow-Managementsystemen, Visualisierung in Monitoring-Systemen, Dienste im Sinne von SOA usw.

die Nachricht zugreifen und diese verarbeiten. Im Java-Umfeld kommt hierfür typischerweise JMS (Java Message Service) zum Einsatz. Der zugehörige Source-Code im Aktionsteil einer Regel sieht etwa wie folgt aus: `QSender.send(event)`, wobei `QSender` eine Klasse ist, die die Verbindung zu einer *JMS-Queue* herstellt und *JMS-Nachrichten* per `send()` verschicken kann.

- Der technisch einfachste Fall ist ein *direkter Dienstaufruf* in der Form `call MyEventHandler.react(..)` im Aktionsteil der Regel. Diese Anweisung ruft für die Klasse `MyEventHandler` die Methode `react()` auf und übergibt den Ereigniskontext als Parameter.
- Statt eines direkten Dienstaufrufs lässt sich auch ein *indirekter Benachrichtigungsdienst* verwenden. Dabei wird beim Ereignismuster ein Listener registriert, der eine festgelegte Methode (beispielsweise `action()`) implementiert. Jedes Mal wenn die Regel feuert, ruft die Event Processing Engine automatisch diese Methode auf. Einen solchen indirekten Mechanismus verwendet zum Beispiel das CEP-Werkzeug Esper (siehe Abschn. 3.3).

3.3 Ereignisregeln mit Esper

Am Beispiel der Open-Source-CEP-Engine *Esper* zeigt dieser Abschnitt, wie eine CEP-Anwendung mit einer konkreten Realisierungsplattform implementiert werden kann.⁶ Die Esper-Plattform stellt eine leistungsfähige und weit verbreitete Event Processing Engine mit einer spezifischen Regelsprache zur Ereignisverarbeitung bereit. Als *Event Processing Language* bietet Esper eine SQL-ähnliche Abfragesprache für Ereignisströme an, die *Esper Query Language* (EQL). Damit gehört Esper zur Klasse der *Continuous Query Languages* (CQL) [2].

Die Ereignisregeln von Esper weisen eine Grundstruktur wie im folgenden Beispiel auf:

```
1 select * from StockEvent where symbol='ibm' and price>100
```

Dieses Muster selektiert alle `StockEvent`-Ereignisse, die sich auf IBM-Aktien beziehen und einen Wert größer als 100 aufweisen. Deutlich erkennbar ist die Ähnlichkeit zu einer SQL-Datenbankabfrage mit den folgenden Elementen:

⁶ Dieser Abschnitt beschränkt sich auf die Vorstellung weniger, grundlegender Sprach-elemente von Esper. Für ausführlichere Informationen sei auf <http://esper.codehaus.org/> verwiesen.

- *From-Klausel*

Die *From-Klausel* legt fest, welche Ereignistypen betrachtet werden. Im Beispiel sind dies Ereignisse vom Typ `StockEvent`. Die Menge der betrachteten Ereignisse kann durch Sliding Windows weiter eingeschränkt werden.

- *Select-Klausel*

Die *Select-Klausel* definiert, welche Ereignisattribute selektiert werden; `*,*` bedeutet, dass alle Attribute des Ereignistyps betrachtet werden.

- *Where-Klausel*

In der *Where-Klausel* können die Ereignistypen noch weiter eingeschränkt werden. Im Beispiel müssen die Attribute `symbol` und `price` entsprechende Werte besitzen.

Den *Aktionsteil* einer Regel realisiert Esper durch ein *Subscriber*-Objekt, das bei dem Muster registriert wird. Jedes Mal wenn die Regel feuert, wird in dem Subscriber-Objekt die dort implementierte Methode `update()` aufgerufen. Der folgende Java-Code zeigt die Implementierung der Subscriber-Klasse.

```

1 public class MyPatternSubscriber{
2     public void update(StockEvent evt){
3         System.out.println(evt.getSymbol() + " " + evt.getPrice());
4     }
5 }

```

Jedes `StockEvent`-Ereignis, das das Muster erfüllt, wird dann als aktueller Parameter an die `update()`-Methode übergeben und kann wie ein ganz normales Java-Objekt verwendet werden.

Ereignismuster in Esper

Auch in EQL ist es möglich, komplexe *ereignistypbasierte* Muster zu beschreiben, und zwar mithilfe des Sprachkonstrukts `pattern[ ]`. Das folgende Beispiel veranschaulicht die Syntax.

```

1 select s.symbol, n.description, s.time-n.time
2 from   pattern[n=NewsEvent -> s=StockEvent(symbol = n.symbol)]
3 where  s.symbol='ibm'

```

Ereignistypen und Aliasnamen Die *From-Klausel* bezieht sich in dem Beispiel auf das im *pattern*-Teil spezifizierte ereignistypbasierte Muster. Das Muster benötigt zwei Instanzen der beiden Ereignistypen `NewsEvent` und `StockEvent`, für die *Aliasnamen* `n` und `s` eingeführt werden.

Sequenzoperator Im Pattern legt der *Sequenzoperator* -> die zeitliche Reihenfolge der Ereignisse fest: n muss vor dem Ereignis s stattfinden. Schließlich muss noch eine Bedingung für das Ereignis s gelten: Der Wert des Attributs `symbol` muss mit dem Wert von `symbol` im Ereignis n übereinstimmen.

Kontextbedingungen Der Kontext der auslösenden Ereignisse lässt sich an zwei Stellen im Ereignismuster festlegen:

1. Innerhalb des Patterns kann eine Bedingung für einen Ereignistyp spezifiziert werden (hier: `StockEvent(symbol = n.symbol)`).
2. Weitere Kontexte lassen sich in der Where-Klausel festlegen. Im Beispiel sollen nur `StockEvent`-Ereignisse betrachtet werden, die einen bestimmten Attributwert enthalten (`s.symbol='ibm'`).

Boolesche Operatoren Darüber hinaus stehen auch die anderen in Abschn. 3.1 eingeführten Sprachkonzepte zur Verfügung. Mit booleschen Operatoren lassen sich Ereignistypen verknüpfen.

- *ODER- und UND-Operator*

Das folgenden Muster zeigt ein Beispiel für eine *ODER*-Verknüpfung. Sobald im Ereignisstrom ein `NewsEvent`- oder ein `StockEvent`-Ereignis auftritt, feuert das Pattern.

```

1  select  s,n
2  from    pattern[n=NewsEvent or s=StockEvent]
```

Alternativ kann natürlich auch ein *logisches UND* verwendet werden. Das folgende Pattern erlaubt, dass in beliebiger Reihenfolge ein `NewsEvent`- und ein `StockEvent`-Ereignis der IBM-Aktie auftreten. Sobald im Ereignisstrom zwei solche Ereignisse eingetroffen sind, feuert das Pattern.

```

1  select  s,n
2  from    pattern[n=NewsEvent and s=StockEvent(symbol='ibm')]
```

- *Negationsoperator*

Der *NOT*-Operator drückt aus, dass ein bestimmtes Ereignis *nicht* stattgefunden hat. Das folgende Pattern beschreibt zwei nacheinander stattfindende `StockEvent`-Ereignisse $s1$ und $s2$, die sich auf dasselbe Wertpapier beziehen und eine Preissteigerung von 20% aufweisen. Gleichzeitig wird verlangt, dass es keine Nachricht zwischen dem Auftreten von $s1$ und $s2$ gibt.

```

1  select  s1 , s2
2  from    pattern [ s1=StockEvent ->
3             ( s2=StockEvent(symbol=s1.symbol , price > 1.2*s1.price )
4             and not NewsEvent(symbol=s1.symbol) ) ]

```

Sliding Windows und Aggregationen Esper stellt umfangreiche Sprachkonstrukte zur Definition von *Sliding Windows* und eine Reihe von *Aggregationsfunktionen* zur Verfügung. Das folgende Ereignismuster definiert ein *Längenfenster* der Größe 3 und summiert das `price`-Attribut der `StockEvent`-Ereignisse auf.

```

1  select  sum(price)
2  from    StockEvent.win: length(3)

```

Das nächste Muster betrachtet das *Zeitfenster* `StockEvent.win:time(200 days)`, also alle `StockEvent`-Ereignisse der letzten 200 Tage mit dem Wert `symbol='ibm'`. In diesem Zeitraum werden die IBM-Aktien selektiert und deren Durchschnittspreis berechnet.

```

1  select  avg(price)
2  from    StockEvent(symbol='ibm').win: time(200 days)

```

Aktionen in Esper

Analog zu den Ausführungen in Abschn. 3.2 können auch in Esper beim Eintreten eines Musters zwei Arten von Aktionen ausgelöst werden:

1. Erzeugen neuer Ereignisse

Das Erzeugen neuer Ereignisse erfolgt mithilfe der *Insert-Klausel*.

```

1  insert into MovingAverageEvent(symbol , average)
2      select  symbol , avg(price)
3      from    StockEvent.std: groupby(symbol).win: time(200 days)
4      group by symbol

```

Die Insert-Klausel erzeugt jedes Mal, wenn das zugehörige Ereignismuster aus den Zeilen 2-4 feuert, ein neues Ereignis vom Typ `MovingAverageEvent` mit den beiden Attributen `symbol` und `average`. Die Group-by-Klausel sorgt dafür, dass der Durchschnitt gemäß der Gruppierung für jedes Symbol getrennt berechnet wird.

2. *Anstoßen von Diensten*

Das Auslösen eines beliebigen Dienstes wird in der `update()`-Methode der Subscriber-Klasse realisiert, beispielsweise über den synchronen Aufruf einer Java-Methode oder durch das asynchrone Senden einer JMS-Nachricht.